

Exercício de Implementação: Sistema de Estoque de Produtos

Java — Classes Abstratas, Herança, Interfaces, Polimorfismo, Composição e Exceções

Entrega

Este exercício deve ser entregue via GitHub: suba o repositório com o código-fonte completo e envie o link do repositório (ou do commit final) conforme instruído pelo professor.

Contexto

Uma loja precisa de um programa em Java que cadastre produtos de tipos diferentes (comuns e perecíveis), controle a venda de itens do estoque e trate corretamente situações inválidas — tudo isso usando uma hierarquia de classes bem desenhada, não apenas ifs soltos.

Requisitos

1. Hierarquia de Exceções

Crie uma exceção base e duas específicas que herdam dela:

```
public class EstoqueException extends Exception {
    public EstoqueException(String mensagem) {
        super(mensagem);
    }
}

public class QuantidadeInvalidaException extends EstoqueException {
    public QuantidadeInvalidaException(String mensagem) {
        super(mensagem);
    }
}

public class ProdutoIndisponivelException extends EstoqueException {
    public ProdutoIndisponivelException(String mensagem) {
        super(mensagem);
    }
}
```

2. Classe abstrata Product

- Atributos encapsulados (private): nome (String), preco (double) e quantidade (int).
- O construtor deve lançar QuantidadeInvalidaException se preco ou quantidade forem negativos.
- Um método abstrato: double calcularValorTotal(); — cada subclasse decide como calcular.
- Um método concreto getDescricao(), que devolve nome, preço e quantidade formatados.
- Implemente a interface Vendavel (item 4) nesta classe.

3. Duas subclasses (herança + polimorfismo dinâmico)

- ProdutoComum extends Product — calcularValorTotal() devolve preco × quantidade, sem regra especial.

- `ProdutoPercivel` extends `Product` — tem um atributo a mais, `diasParaVencer` (int). Sobrescreve (`@Override`) `calcularValorTotal()` aplicando 20% de desconto automático quando `diasParaVencer` $\leq$ 3. Sobrescreve também `getDescricao()` para incluir a validade.

4. Interface Vendavel

Crie a interface e implemente-a em `Product`:

```
public interface Vendavel {
    void vender(int quantidadeDesejada)
        throws ProdutoIndisponivelException;
}
```

O método `vender()` deve lançar `ProdutoIndisponivelException` se `quantidadeDesejada` for maior que o estoque disponível; caso contrário, deve subtrair a quantidade vendida.

5. Sobrecarga: aplicarDesconto() (polimorfismo estático)

Adicione à classe `Product` as duas versões do método (mesmo nome, assinaturas diferentes):

```
void aplicarDesconto(double percentual) { ... }
void aplicarDesconto(double percentual, double descontoMaximo) { ... }
```

6. Classe Estoque (composição)

- A classe `Estoque` TEM UMA lista (ou array) de `Product` — não herda de `Product`.
- `adicionarProduto(Product p)` — adiciona um produto à lista.
- `venderProduto(int indice, int quantidade)` — chama `vender()` do produto correspondente, propagando `ProdutoIndisponivelException` se ocorrer.
- `calcularValorTotalEstoque()` — percorre a lista somando o resultado de `calcularValorTotal()` de cada produto (é aqui que o polimorfismo aparece: cada produto calcula do seu próprio jeito, sem o `Estoque` precisar saber qual subtipo é).

7. Classe EstoqueApp (com main())

- Cadastre pelo menos 2 `ProdutoComum` e 2 `ProdutoPercivel` (um deles com `diasParaVencer` $\leq$ 3).
- Tente cadastrar um produto com quantidade negativa, capturando `QuantidadeInvalidaException`.
- Venda uma quantidade válida de um produto, e depois tente vender mais do que o disponível, capturando `ProdutoIndisponivelException`.
- Imprima o valor total do estoque, mostrando que produtos comuns e perecíveis são somados corretamente mesmo com regras de cálculo diferentes.

Pista sobre a ordem dos catches

Se você capturar `QuantidadeInvalidaException`, `ProdutoIndisponivelException` e `EstoqueException` no mesmo bloco `try`, o `catch` de `EstoqueException` (o mais genérico) precisa vir por último — colocá-lo antes torna os `catches` seguintes inalcançáveis.

O Que Entregar

- ☐ Todas as classes e interfaces (EstoqueException, QuantidadeInvalidaException, ProdutoIndisponivelException, Product, ProdutoComum, ProdutoPercivel, Vendavel, Estoque, EstoqueApp), compilando sem erros.
- ☐ A saída do programa mostrando: os produtos cadastrados, o valor total do estoque, as duas exceções sendo capturadas corretamente, e uma venda bem-sucedida.